

Lab 3: DelayPurpose

Measure PIC16F883 instruction timing, use instruction cycles calcs, create software delays with nested loops.

Standards & References

RCET 3375 Lab Standards
 RCET PIC-AS Style Guide
 PIC16F883 Datasheet
 PIC micro mid range new Family Ref Manual

Equipment & Materials

- MPLAB & PIC-AS toolchain
- PICkit
- PIC16F883
- 4MHz crystal oscillator
- DLG7137 LED display
- Oscilloscope
- DMM
- Lab book

Part 1 - Measure instruction timing

Goal - Use port B counter to measure execution time

Use RBO to:

- measure pulse width, space, period, & frequency
- relate measured waveform to the instructions executing in program
- determine actual instruction cycle time, T_{CY}
- calculate oscillator frequency
- verify the effect of adding & removing NOP operations

Before Lab

Start with working PORTB counter program

For each instruction in the repeating path:

1. Identify how many instruction cycles it requires
2. Determine how many instruction cycles occur between RBO changes

$$T_{OSC} = \frac{1}{F_{OSC}} \rightarrow T_{CY} = 4T_{OSC} \rightarrow F_{OSC} = 4/T_{CY}$$

$$1/4\text{MHz} = 250\text{ns} \quad 250\text{ns} \cdot 4 = 1\mu\text{s} \quad 4/1\mu\text{s} = 4\text{MHz}$$

This circuit uses the 4MHz oscillator.

Determine expected number of instruction cycles for

- RBO PW: 3μs
- RBO PS: 3μs
- one complete RBO period: 6μs

Refer back to page 12

Part 1 - Measure instruction timing

In the Lab:

1. Measure the Lab2 Counter

Connect the oscilloscope to RBO, the least significant bit

Measure & record **Refer back to page 12**

- PW: 2.9 μ s

- PS: 3.1 μ s

- Period: 6 μ s

- Frequency: ~~166.667~~ 166.667 kHz

Capture an oscilloscope image showing enough of the waveform to verify
Refer back to page 12

Compare PW & PS & explain why RBO changes at the observed rate based on the binary count in the main loop

PW & PS here are just a 50% duty cycle, because measuring RBO is measuring the first part of a binary count which only switches between high & low at 50%. This count is determined by the equation that gets TOSC, time of oscillation, then multiply that by 4 to get the actual instruction time. Looking back at page 11, with the code in main, (being just the increment & go to main) there's three instructions. That means PS & PW = 3 μ s, so period = 6 μ s.

2. Determine the instruction cycle time

Use the measured waveform & instruction cycle analysis of main loop to calculate actual instruction time

N = instruction cycles occur during one measured interval

$$TCY = \frac{\text{measured}}{N}$$

$$TCY = \frac{6 \mu s}{6 \text{ instructions}} \quad TCY = 1 \mu s$$

3. Determine oscillator frequency

Calculate the actual oscillator frequency from measured instruction cycle time

$$FOSC = 1/TCY$$

$$FOSC = 1/1 \mu s$$

$$FOSC = 1 \text{ MHz}$$

Compare to actual 4 MHz

$$\frac{F_{\text{measured}} - F_{\text{nominal}}}{F_{\text{nominal}}} \times 100$$

$$\frac{1 \text{ MHz} - 4 \text{ MHz}}{4 \text{ MHz}} \times 100 = 0$$

4. Add & remove Nop

Add one Nop instruction to the repeating execution path

Before programming, predict change in PW, PS, period, & frequency

Build & compare

Remove Nop & reprogram

Lab 3 Delay

Part 1 Measure Lab 2 counter

Predictions for adding one Nop

PW: 4 NS

PS: 4₁₀₅

Period: 8ms

Frequency: 125kHz

Build 3: program the modified code

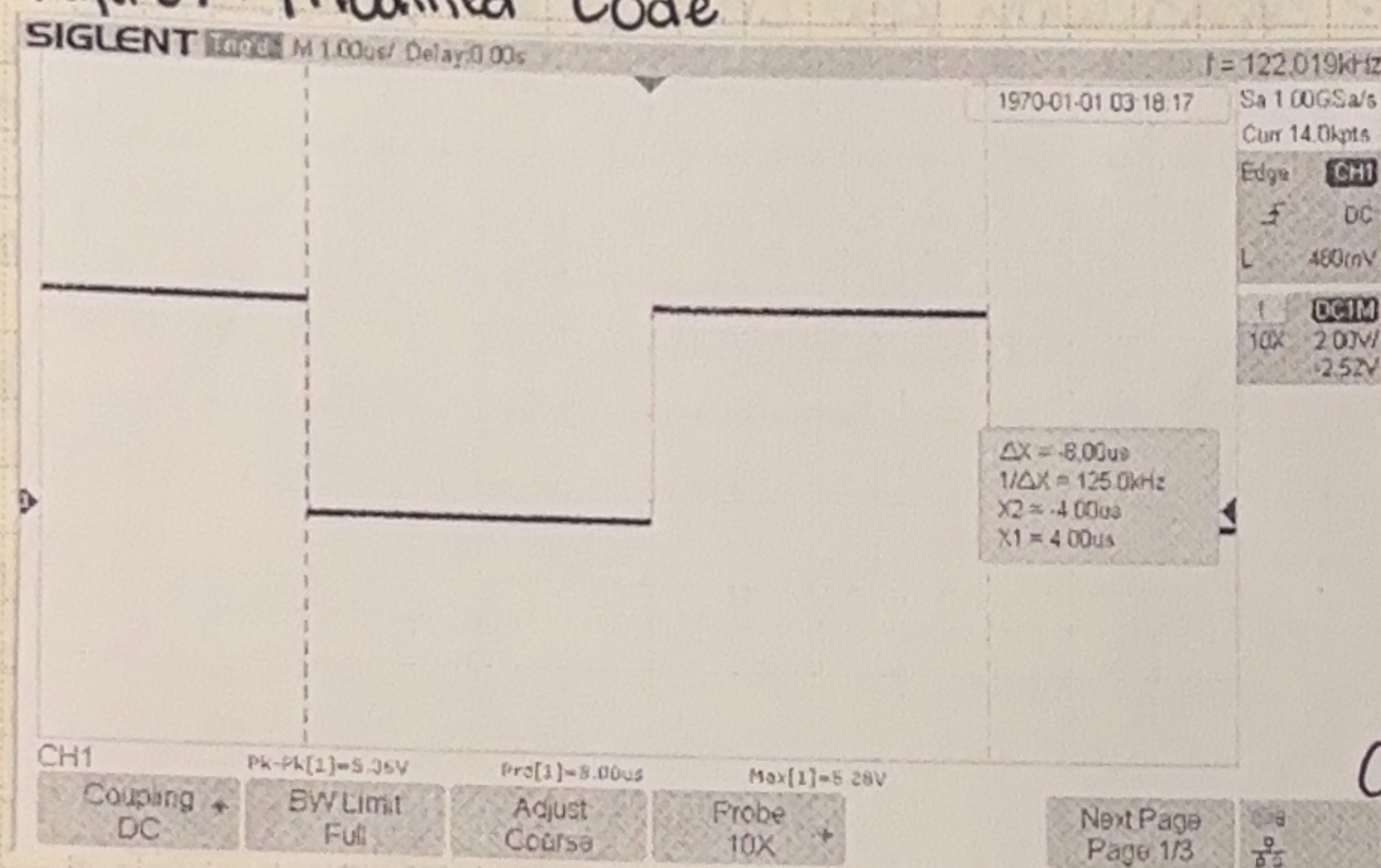
```

87 Main:
88 //main code goes here. Repeats forever because setup goes once
89 NBP
90 INCF 0x06,1 ;plus 1 to port B forever
91
92 goto Main
93 End

```

am

Figure 1: Modified Code



With the modified code, an extra 1 μ s is added, leading to a period of 8 μ s total. This drops the frequency to 125 kHz.

An

Figure 2: Modified code measured

Part 2 Create a Precise Short Delay

Goal - Create a PIC16F883 project to toggle RBO using XORMF then add a calculated 10KHz delay

Before Lab:

Create a new project using the previous code

Add

Main:

```
movlw 0x01; 1TCY
```

XORMF PORT 13, 1; 1 TCY

GOTO Main ; ZTCY

Determine

PW: 4 ps

PS: 4, 2, 5

Period: 8th NS

Frequency: 120 kHz

Duty Cycle: 50%

Lab 3 DelayPart 2 Create a Precise Short Delay

In the Lab:

1. Verify the XORWF toggle loop
- Build & program the new project
- Measure RBO on oscilloscope

Record:

PW: $4\mu s$ PS: $4\mu s$ Period: $8\mu s$ Frequency: $125kHz$ Duty cycle: 50%

This came out the same as the last part of part 1.

The calculations & measurements come out the same, because the microcontroller is so specific with its timing.

2. Determine the required timing
- Modify the program so RBO produces:

- $10kHz$ - 50% duty cycle

$$T = 1/10K = 10\mu s \text{ period}$$

$$Pw = PS = T/2 = 5\mu s$$

$$TCY = 1\mu s$$

Determine the number of instruction delay cycles required between successive RBO transitions. Account for every instruction executed between transitions. Do not treat the delay loop as if it were the only loop consuming time.

$$\text{Instruction} = 5\mu s - 4\mu s = 1\mu s$$

For either a PW or PS, each is $5\mu s$ because the device scans twice every cycle. So to get $5\mu s$, you would either need 46 no operation instructions, or to create a nested delay totaling 46.

3. Use Create a Software delay loop

Use an 8-bit counter & DECFSZ to add the delay. Use the following example

```

count EQU 0x20 ;use data memory address 0x20 as an 8-bit loop counter

MainLoop:
    MOVLW 0x01 ;1 TCY code not part of delay
    XORWF PORTB,1 ;1 TCY code not part of delay

    MOVLW 0x09 ;1 TCY delay overhead
    MOVWF count ;1 TCY delay overhead
DelayLoop:
    DECFSZ count,1 ;1(2) 1 TCY if not zero, 2 TCY if zero
    GOTO DelayLoop ;2 TCY

    GOTO MainLoop ;2 TCY code not part of delay
  
```

The XOR is used to toggle PORT B's channel instead of each of the other RB1-7.

CM

Figure 3 : example code

Lab 3 Delay

Part 2 Create a Precise short delay

To solve N:

$$T_{Loop} = N(1+2) - 1 \quad \text{Inner Loop}$$

$$\downarrow T_{Loop} = 3N - 1$$

Now add overhead

$$\downarrow T_{Delay} = N(1+2) - 1 + 1 + 1$$

$$\downarrow T_{Delay} = N(3) + 1$$

Add GOTO to main

$$\downarrow \underline{T_{Delay} = 3N + 5}$$

So using the example where 50ps for PW/PS is desired, ~~N=15~~. NOPs would be used somewhere to make the number whole without going over 50ps.

$$\underline{N=15}$$

4 Determine Counter value

Use $PW = PS = 50\mu s \approx TCY = 1\mu s$

Then determine:

- instruction cycles consumed outside the delay: 4
- instruction cycles required from delay: 1
- required counter value: 15
- whether any NOP instructions are needed

Record desired:

- desired PW & PS: 50ps
- required instruction cycles between transitions: none
- instruction cycles consumed outside delay: 4
- required delay cycles: 1
- Selected counter: 15
- Any included NOP - None needed
- Predicted:
 - PW: 50ps
 - PS: 50ps
 - Period: 100ps
 - Frequency: 10kHz
 - Duty cycle: 50%

Show modified source code with this delay

```

Main:
    //main code goes here. Repeats forever because setup goes once
    MOVLW 0x01                ;1 TCY code not part of delay
    XORWF PORTB,1            ;1 TCY code not part of delay

    MOVLW 0x0F                ;1 TCY delay overhead
    MOVWF count              ;1 TCY delay overhead

DelayLoop:
    DECFSZ count,1           ;1(2) 1 TCY if not zero, 2 TCY if zero
    GOTO DelayLoop          ;2 TCY

GOTO Main                  ;2 TCY code not part of delay
    
```

Figure 4 modified code

Lab 3 Delay Part 2 Create a Precise Short Delay 5. Build & measure the calculated Delay measure RBO with the new modified code

Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter
PW	50ns	50ps	50ps	50.002ps
PS	50ns	50ps	50ps	49.999ps
Period	100ns	100ps	100ps	99.9999ps
Frequency	10kHz	10kHz	10kHz	10.0005kHz
Duty cycle	50%	50%	50%	50.0005%

Table 1: Recorded predictions, desired, & measured numbers.

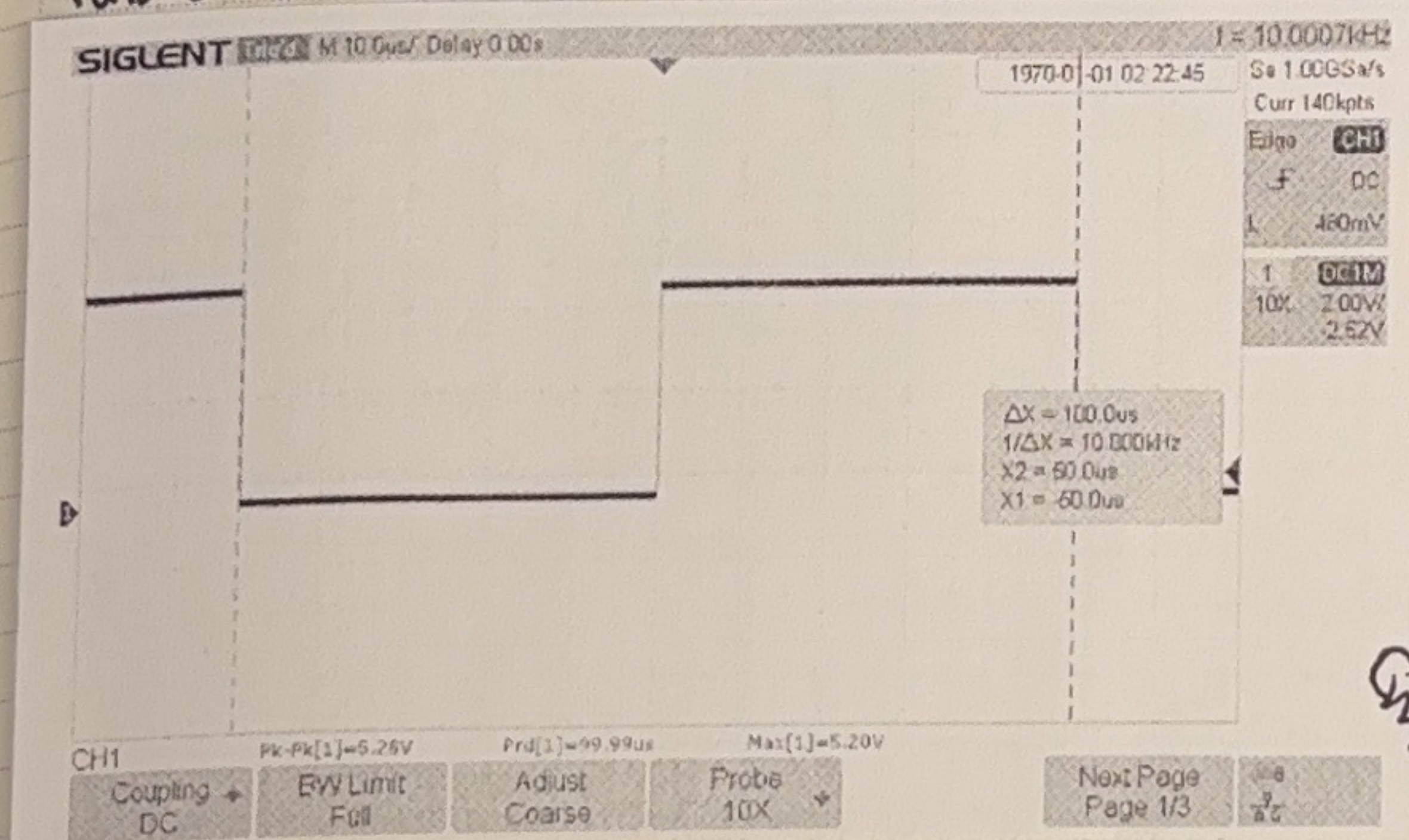


Figure 5 Oscilloscope capture of modified program

Each calculation was accurate to the measurement. Setting the delay to count from 15 gets the required number of cycles/period

TSR
9-9-26

Part 3 - Create & Verify a long delay

Goal - Use nested software to delay loops to create a much longer time interval than can be conveniently made with one 8 bit loop.

$$PW = PS = 0.55$$

Before lab : TCY = 1

Determine longest delay with 1 8 bit loop : ~~255~~ 770

Using the equation from previous to get the max

$$\max = 255(3) + 1 = 770$$

Delay Lab 3

Part 3 - Create a long delay

Before lab:

Design an inline nested delay loop capable of producing 0.5s.

Do not use:

- Subroutines
- timer peripherals
- interrupts

Account for every instruction cycle

Use TCY to predict PS, PW, period, frequency $\approx$ duty cycle

With 0.5s, Frequency = 1Hz DC=50% PS/PW=0.5s

1 nested loop = 770

2 nested loop = $255(255(3)+1)+2 = 195k$

3 nested loop = $255(255(255(3)+1)+2)+2 = 49.8M$

We need 0.5 or ~~5m~~ TCY 500k TCY

Started with the 3 nested loops $\approx$ lowered the numbers to get the needed values.

TCY 500.05k = $(005(100(249(4)+4)+2)+5$

In the lab:

1. Build the nested loop

```
BCF 0x03,6 ;set bank 0 active
BCF 0x03,5 ;set bank 0
OutCount EQU 0x20
MidCount EQU 0x21
InCount EQU 0x22
MOVLW 0x30 ; Setting portC to display the 0
MOVWF 0x07
```

CM

Figure 6 Setup code

```
Main:
    //main code goes here. Repeats forever because setup goes once
    MOVLW 0x05
    XORWF 0x07,1
    MOVLW 0x05
    MOVWF OutCount
OuterDelay:
    MOVLW 0x64
    MOVWF MidCount
MidDelay:
    MOVLW 0xF9
    MOVWF InCount
InDelay:
    NOP
    DECFSZ InCount
    GOTO InDelay

    DECFSZ MidCount
    GOTO MidDelay

    DECFSZ OutCount
    GOTO OuterDelay

    GOTO Main
```

CM

Figure 7 The count nested loop code

Lab 3 Delay
 Part 3 - Create a long delay
 2. Measure with the oscilloscope
 3. Measure with the frequency counter

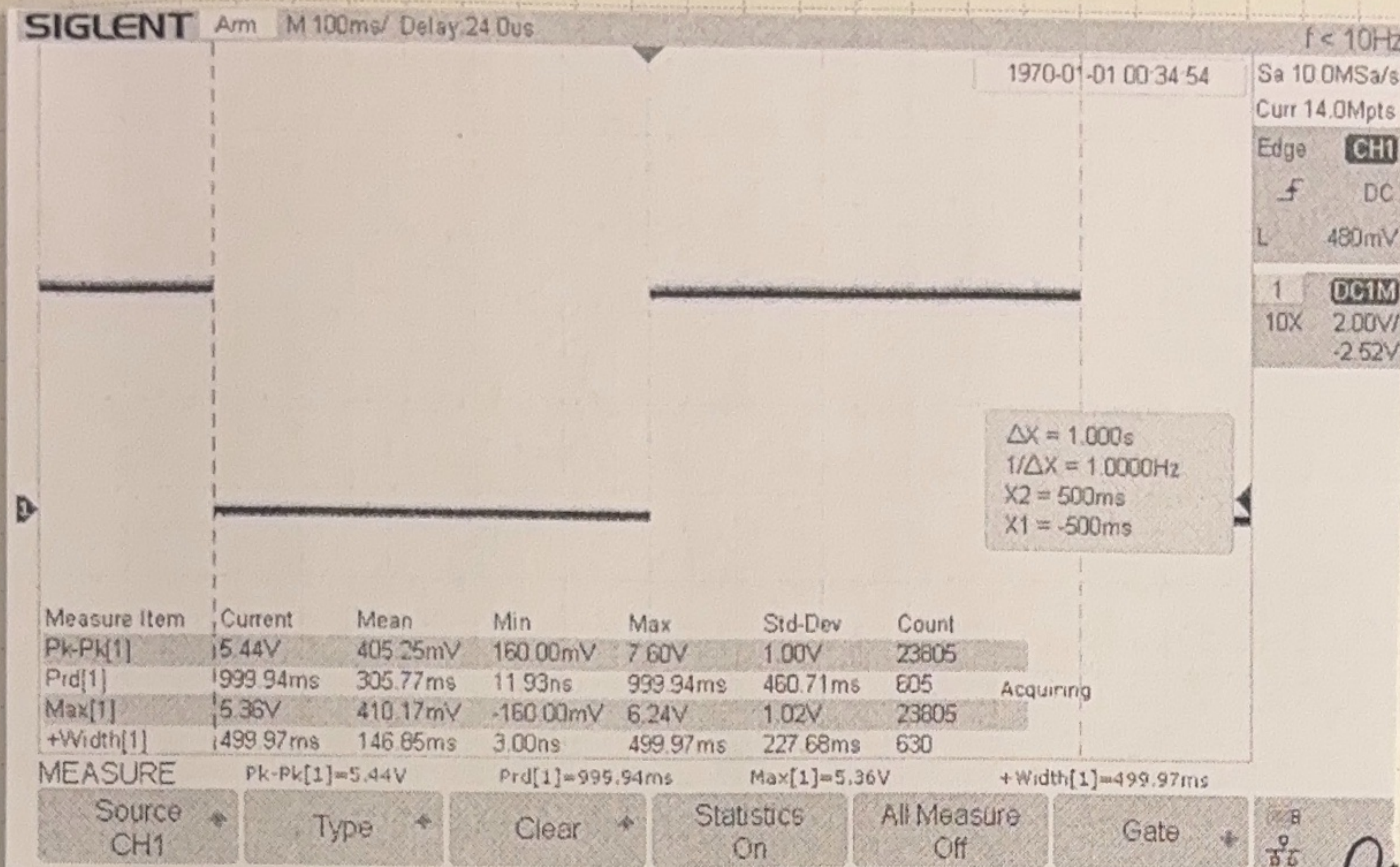


Figure 8 Oscilloscope capture

Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter
PW	500ms	500ms	500ms	500.00ms
PS	500ms	500ms	500ms	500.00ms
Period	1s	1s	1.000s	1.00001s
Frequency	1Hz	1Hz	1Hz	1.0001Hz
Duty cycle	50%	50%	50%	53%

Table 2 Long delay predictions & measurements

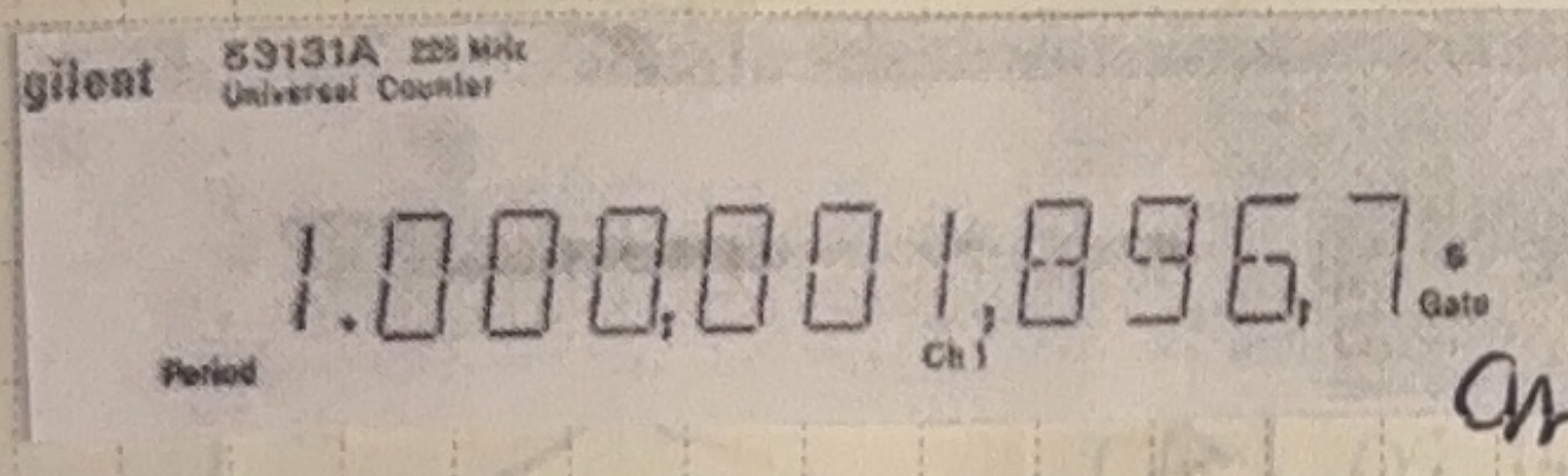


Figure 9 measured period

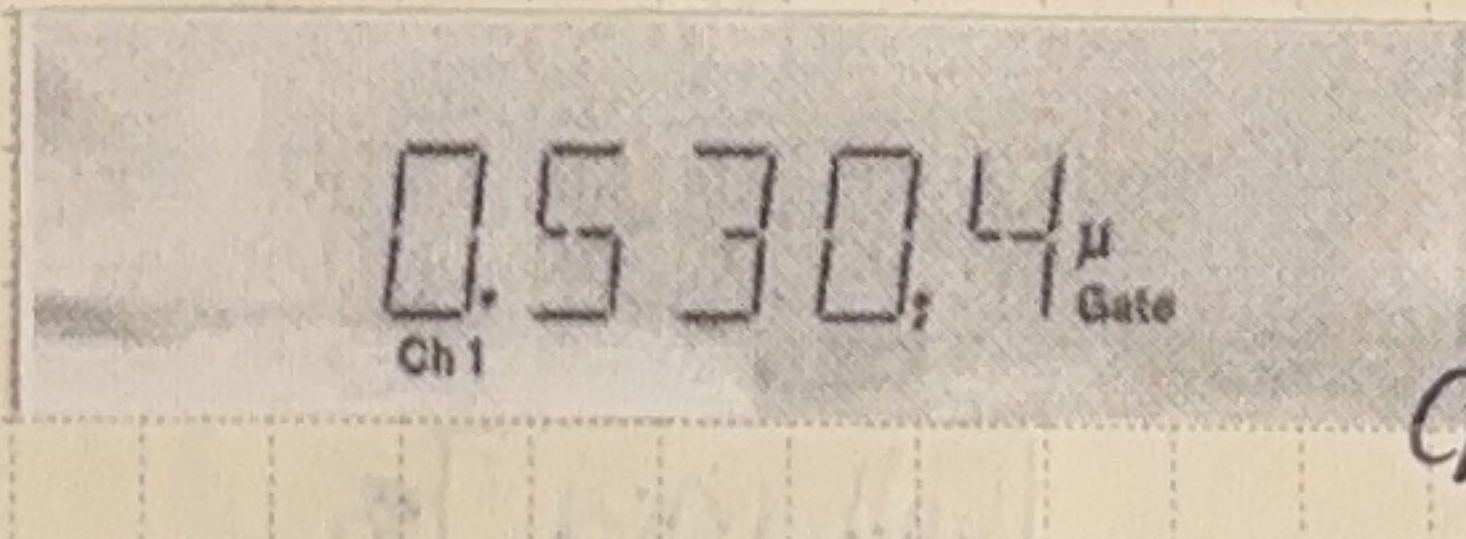


Figure 10 measured duty cycle

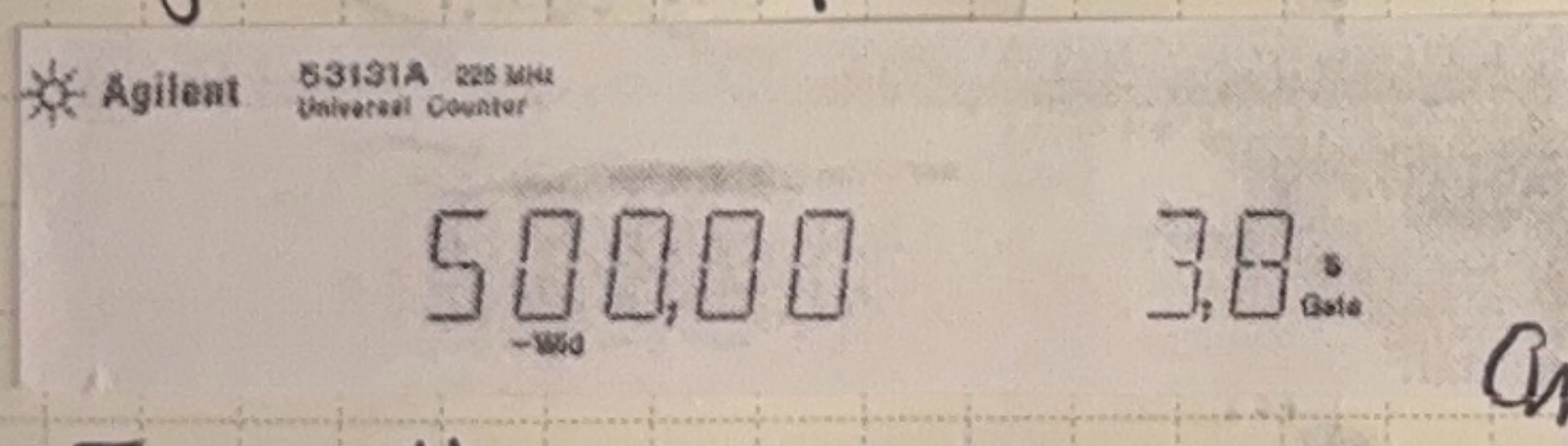


Figure 11 measured pulse space

The oscilloscope gives a lot of really good information & actually lets you see the waveform. The frequency counter is far more accurate & shows more numbers.

Lab 3 Delay

Part 3 - Creating a short delay

4. Change the program by one instruction cycle
Add one NOP to the path between output transitions

Measurement	Before NOP	With NOP	Difference	Instrument Resolution
PW	500001 500ms	500.001ms	0.000001	10 / per sec

cm

Table 3 NOP Added measurements

Adding the NOP to main increased the time by less than 0.000001, it was barely noticeable.

The frequency counter used in lab can measure from 0.1 to 225 MHz. The different settings allow you to be able to fine tune down to measure low frequencies accurately.

Part 4 - Delay Subroutines & the Hardware Stack

Goal - Reuse the previous code but use call/return subroutines

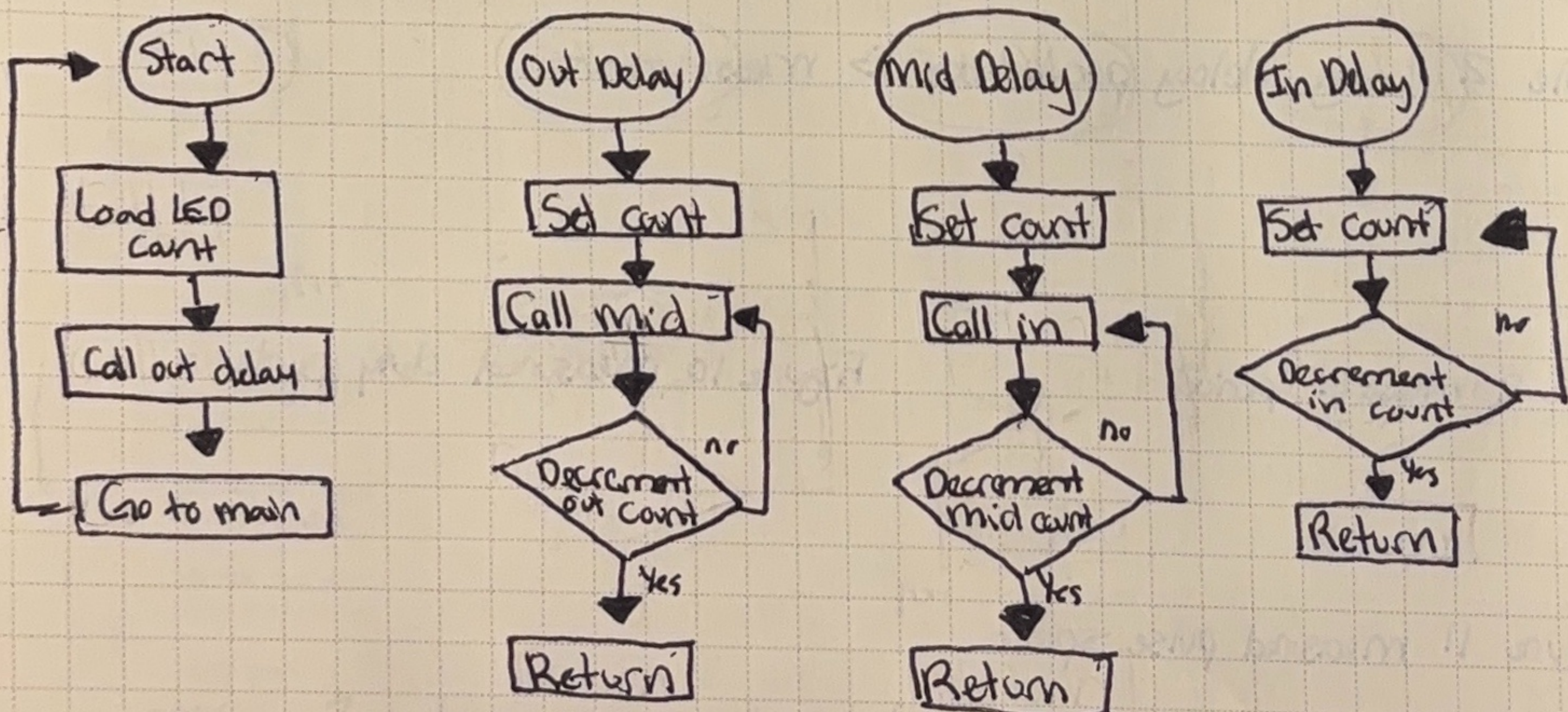
Before lab:

Modify the part 3 code so it contains:

- a main program that alternates the DLR7137 between 0 & 1
- a reusable long delay subroutine
- a shorter delay subroutine called within longer routine
- call & return instructions

9-11-26 Create a flowchart showing the subroutine & calls & returns

Flow Chart



Lab 3 Delay Part 4 - Delay Subroutine & hardware stack In the lab: 1. Implement subroutines 2. Verify timings

Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter
PW	500ms	500ms	499.999ms	499.99905ms
PS	500ms	500ms	500.001ms	500.00095ms
Period	1s	1s	1s	1Hz
Frequency	1Hz	1Hz	1Hz	1Hz
Duty cycle	50%	50%	49.99%	49.6%

Table 4 measured for subroutines



Figure 12 measured period

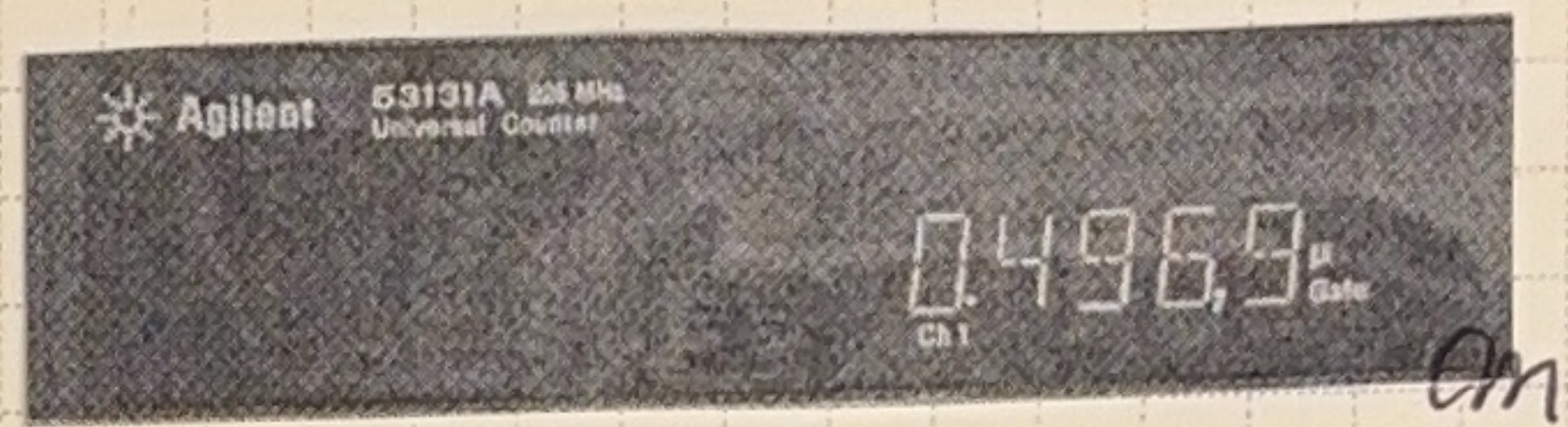


Figure 13 measured duty cycle



Figure 14 measured pulse

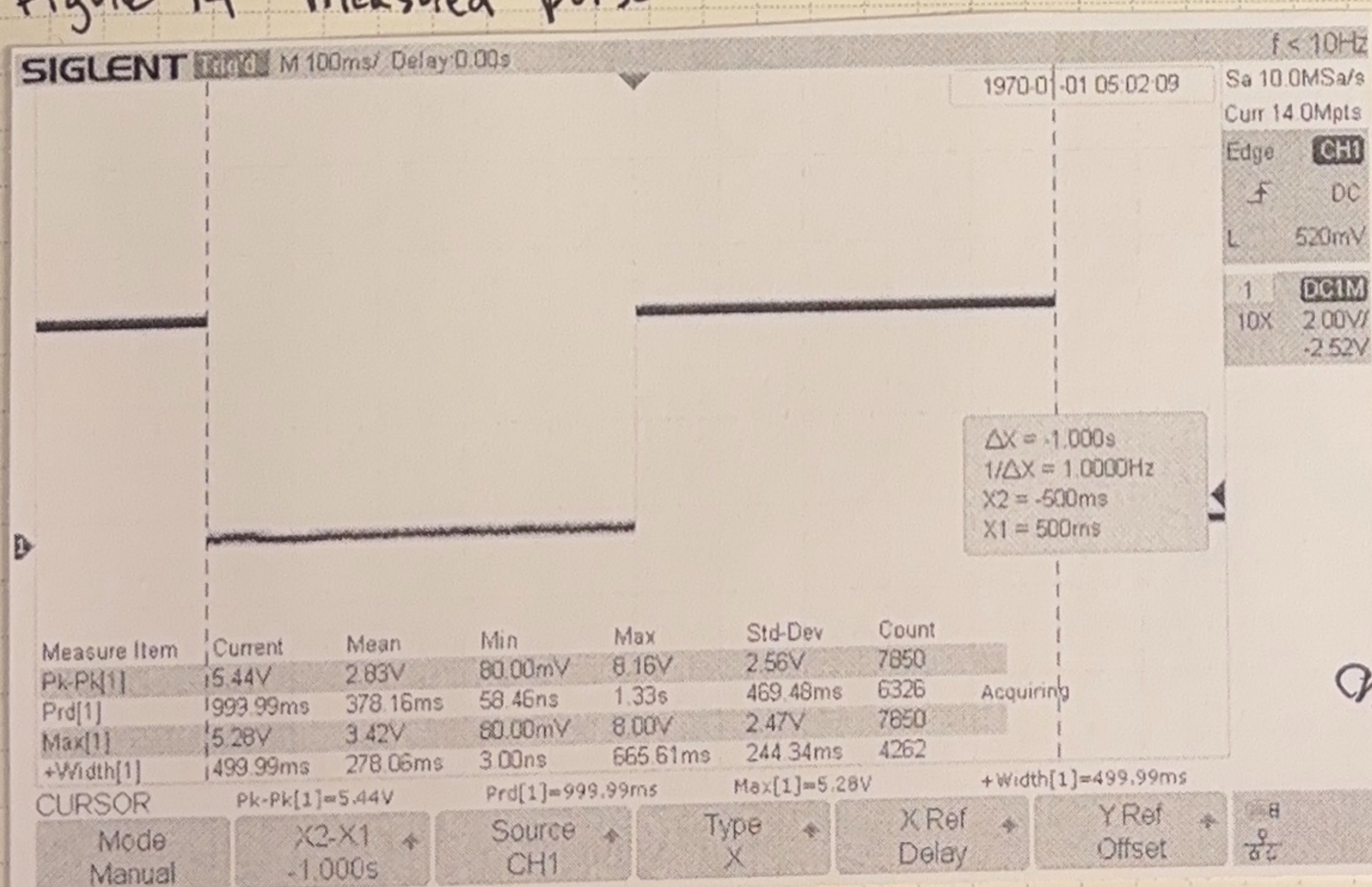


Figure 15 oscilloscope capture

Lab 3 Delay

Part 4- Delay Subroutines

```

Main:
    //main code goes here. Repeats forever because setup goes once
    MOVLW 0x05 :1 TCY
    XORWF 0x07,1 : 1 TCY
    NOP : 1 TCY
    NOP: 1 TCY
    NOP: 1 TCY
    NOP: 1 TCY
    CALL OuterDelay :2 TCY
    GOTO Main : 2 TCY

OuterDelay:
    MOVLW 0x04: 1 TCY
    MOVWF OutCount : 1 TCY

OuterLoop:
    NOP: 1 TCY
    NOP: 1 TCY
    NOP: 1 TCY
    NOP: 1 TCY
    NOP: 1 TCY
    NOP: 1 TCY
    CALL MidDelay : 2 TCY
    DECFSZ OutCount : 1(2) TCY
    GOTO OuterLoop : 2 TCY
    Return : 2 TCY

MidDelay:
    MOVLW 0xB7: 1 TCY
    MOVWF MidCount : 1 TCY

MidLoop:

    CALL InDelay : 2 TCY
    DECFSZ MidCount :1(2) TCY
    GOTO MidLoop : 2 TCY
    Return : 2 TCY

InDelay:
    MOVLW 0xE1 : 1 TCY
    MOVWF InCount : 1 TCY

InLoop:|
    DECFSZ InCount : 1(2) TCY
    GOTO InLoop : 2 TCY
    Return : 2 TCY
  
```

152
9-11-26

Cm

Figure 1b implemented code

3. Trace the nested calls

Main

→ Call out delay
 → Call mid delay
 → Call in delay
 ← return
 ← return
 ← return
 ← return

Lab 3 Delay Part 4 Delay Subroutines

Conclusion

The time of the waveforms on the PIC outputs are determined by the external oscillator & by the instruction cycles. The instruction cycles can be used to make specific time outputs. Delays of various types can be used in many ways to make. A single 8 bit loop can make a counter with 270 instruction cycles. With nested loops, they can only go 9 deep in the call stack. With call & returns, you can make as many subroutines as needed, you just need to remember return will return the code to the previous where the code was called.